End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 — Software Production Engineering · Mini-Project Report M.Tech (CSE), Term I, 2026-27 · Faculty: Prof. B. Thangaraju

Repository: `https://github.com/jayeshpandit/calculator-devops` Docker Hub: `https://hub.docker.com/r/jayeshpandit/calculator-api` (Both names are reserved for this submission; the pipeline references them by `IMAGE_REPO`/`REGISTRY_CREDS` in the `Jenkinsfile`.)

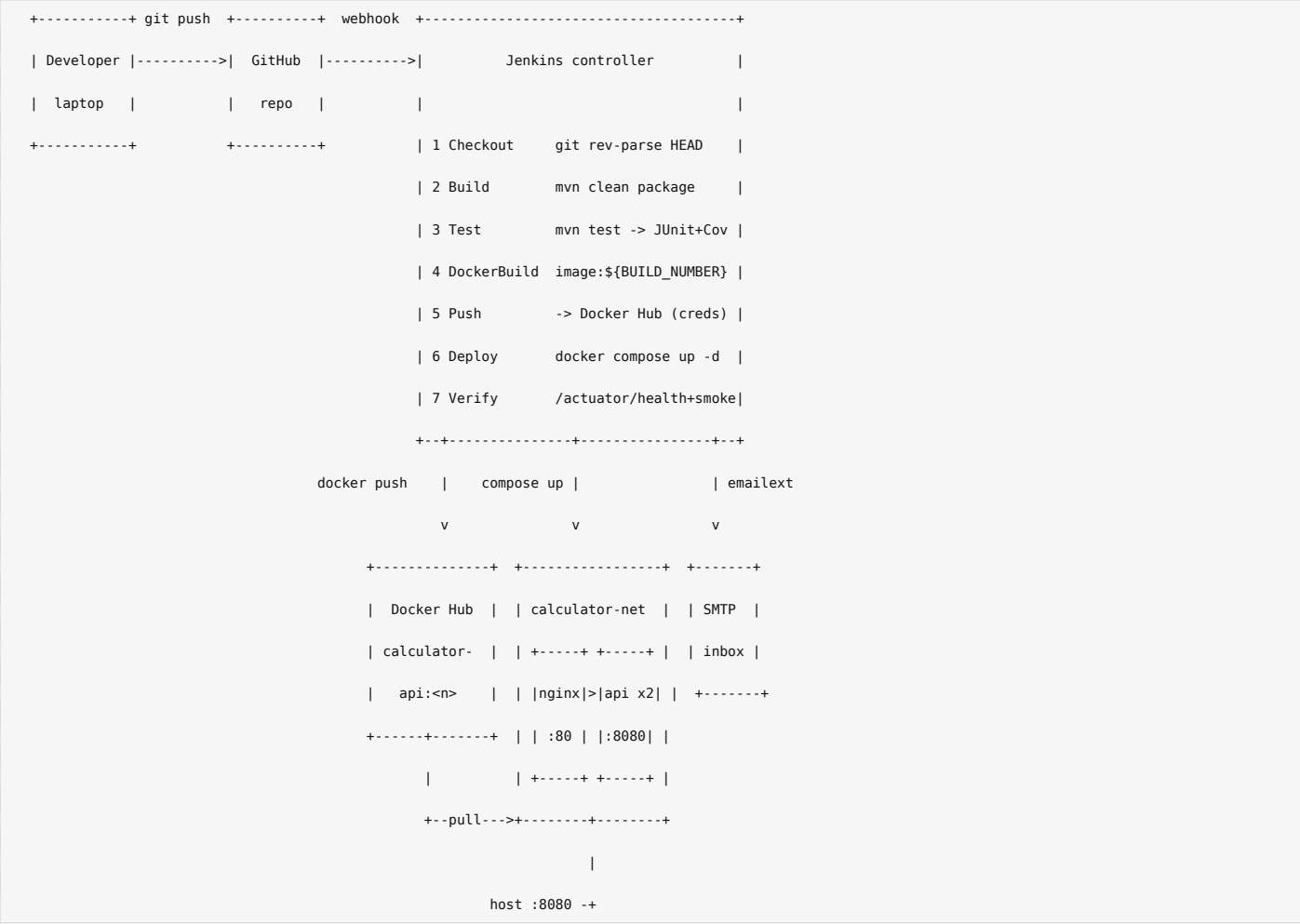
---

1. Overview

The deliverable is a Calculator REST API (Java 21, Spring Boot 3.3, Maven) surrounded by a fully automated delivery pipeline. The application is intentionally small — four arithmetic endpoints and an Actuator health probe — so that the engineering effort sits in the pipeline: a `git push` to `main` triggers Jenkins over a GitHub webhook, and seven pipeline stages take the change from source to a redeployed, health-verified, load-balanced container stack without a single manual step.

Four tools are in scope, matching the pre-midterm modules: **Git/GitHub** for version control, **Jenkins** for CI/CD, **Docker** for containerization and **Docker Compose** for deployment.

2. Pipeline architecture



Runtime topology. Two API replicas join a user-defined bridge network and publish no host ports; all traffic enters through an Nginx reverse proxy on host port 8080. The proxy resolves the service name against Docker's embedded DNS on every request (see challenge 7), so it both round-robins across the replicas and follows them across a redeploy, retrying the sibling on `502/503/504`. Each replica carries a health check that greps `"status":"UP"` out of `/actuator/health`, and the proxy only starts once the API reports `service_healthy`.

3. Stage-by-stage description

#	Stage	Implementation	Why it matters
1	**Checkout**	`checkout scm`; captures short SHA and commit subject into the build environment	Every artefact and notification is traceable to one commit
2	**Build**	`./mvnw -B clean package -DskipTests`; archives `target/calculator.jar` with fingerprinting	Fails fast on compilation errors before the slower test stage
3	**Test**	`./mvnw -B test`; `post{always}` publishes `target/surefire-reports/*.xml` via the `junit` step and `jacoco.exec` via the JaCoCo plugin	A red test fails the build; reports are published even for failed builds
4	**Docker Build**	`docker.build("${IMAGE_REPO}:${BUILD_NUMBER}", '--pull .')`, also tagged `latest`	Immutable, build-number-addressable images; `--pull` keeps base images fresh
5	**Push**	`docker.withRegistry(..., 'dockerhub-credentials')` pushes both tags	Credentials come from the Jenkins store — never from the repository
6	**Deploy**	`docker compose pull` then `docker compose up -d --remove-orphans`	Compose reconciles the running stack in place, which is what makes re-runs idempotent
7	**Verify Health**	polls `/actuator/health` up to 30× / 5 s, then asserts `add`/`multiply` and every error path through the proxy: four 400s, a 404 and a 405	Deployment is only "green" if the new containers actually serve traffic correctly
—	**post**	prunes dangling images older than a week, deliberately leaving the deployed tag alone; `emailext` sends HTML mail on success / failure (log attached) / unstable	Disk hygiene without stranding the running stack on a missing image

`options` adds `timestamps()`, a 30-minute timeout, log rotation (15 builds), and `disableConcurrentBuilds()` — the last matters because every build deploys to the same host, so two overlapping builds would race over port 8080.

4. Application and test design

The arithmetic lives in `CalculatorService`, free of any web concern, so it is unit-testable in isolation. `CalculatorController` is a thin adapter returning a `CalculationResponse` record (`{"operation","a","b","result"}`). Every failure mode is funnelled through one `@RestControllerAdvice`:

Condition	HTTP	`error` field
`b = 0` on divide	400	`Division by zero`
missing `a` or `b`	400	`Missing parameter`
non-numeric parameter	400	`Invalid parameter`
`NaN`/`Infinity` operand, or a result that overflows	400	`Invalid parameter`
unknown path	404	`Not Found`
wrong HTTP method	405	`Method Not Allowed`
anything unexpected	500	`Internal Server Error`

This matters for the pipeline, not just for tidiness: the Verify stage asserts that `divide?a=1&b=0` answers 400 — and likewise that an unknown path is 404 and a wrong method 405 — so a regression that lets `b=0` return `Infinity` (the Java default for double division), or that collapses routing errors into 500s, breaks the deployment gate rather than reaching users.

Tests: **35 total, all passing.** Parameterized JUnit 5 cases cover all four operations plus the `x/0` and `0/0` rejections and the non-finite guards; a `@SpringBootTest` + `MockMvc` suite drives every real endpoint, every 4xx path including 404 and 405, and the Actuator probe; a context test asserts the wiring.

5. Container and image design

The `Dockerfile` is multi-stage. Stage `build` (`maven:3.9-eclipse-temurin-21`) copies only `pom.xml` first and runs `dependency:go-offline`, so editing Java source re-uses the cached dependency layer instead of re-downloading the Maven world. Stage `runtime` (`eclipse-temurin:21-jre-alpine`) copies nothing but `calculator.jar` — the JDK, Maven and the entire `~/.m2` cache stay behind in the discarded builder.

Security and hygiene: a dedicated non-root user (`appuser`, uid 10001) owns the JAR and runs the JVM; `EXPOSE 8080` is explicit; `HEALTHCHECK` gives Docker a real liveness signal; OCI labels describe provenance; and `.dockerignore` keeps `target/`, `.git/`, docs and any `.env` out of the build context (smaller context, and no chance of a stray secret landing in a layer). `JAVA_OPTS` defaults to `-XX:MaxRAMPercentage=75`, so the JVM heap respects the Compose memory limit instead of sizing itself against the host.

6. Git practice

`main` is the integration branch; work arrives through short-lived feature branches merged with `--no-ff`, so the graph permanently records each merge:

`feature/calculator-api` → `feature/unit-tests` → `feature/docker` → `feature/compose-deployment` → `feature/jenkins-pipeline` → `docs/report`

Commits are small and progressive across the work window rather than a single drop. The `.gitignore` excludes `target/`, IDE directories, logs and every secret-shaped path (`.env`, `*.pem`, `*.key`, `*.jks`, `credentials.json`); `docker-compose.override.yml` is ignored so local experiments cannot be committed by accident. No credential appears anywhere in the history — the Docker Hub login is a Jenkins credential referenced only by ID.

7. Challenges faced

1 — Publishing test reports from a failing build. Putting `junit` in the stage's `steps` meant it never ran when `mvn test` exited non-zero: the build went red with no visible reason. Moving both publishers into `post { always { … } }` keeps the failure *and* surfaces which test broke.

2 — Idempotent redeploys. The first Deploy stage ran `docker compose down` before `up`, which produced a window of downtime on every build and left orphaned containers when a service was renamed. Switching to `docker compose up -d --remove-orphans` lets Compose reconcile only what actually changed; combined with `disableConcurrentBuilds()`, a re-run of an unchanged build is a genuine no-op.

3 — Health check that lies. `curl` is absent from the Alpine JRE image, and `wget -qO- /actuator/health` exits 0 on any HTTP 200 — including a `"status":"DOWN"` body. The check now pipes through `grep -q '"status":"UP"'`, so a degraded-but-serving container is correctly reported unhealthy. `start_period: 30s` stops Spring Boot's startup time from being counted as a failure.

4 — Replicas versus published ports. Giving the replicated service a `ports:` mapping made the second replica fail to start on an "address already in use" error. The fix was architectural rather than cosmetic: replicas `expose` the port on the internal network only, and the proxy is the single thing bound to a host port — which is precisely what the Nginx component is for.

5 — JVM memory inside a container. The container was OOM-killed under the 512M Compose limit because the JVM sized its heap from host RAM. `-XX:MaxRAMPercentage=75` makes it respect the cgroup limit.

6 — Docker-in-Jenkins permissions. `docker build` from the pipeline failed with a socket permission error until the `jenkins` user was added to the `docker` group and the service restarted.

7 — The proxy cached dead replica IPs. The first `nginx.conf` used an `upstream` block naming `calculator:8080`. Open-source Nginx resolves an upstream host *once*, at config load. Round-robin worked, so the bug was invisible — but on the next deploy the replicas were recreated with new IPs while Compose correctly left the unchanged `proxy` alone, so Nginx kept dialling the old addresses and served 502s until restarted by hand. Because every peer was stale, `proxy_next_upstream` could not rescue it, and `Verify Health` would have failed the first build that actually changed the image. The fix is to point `proxy_pass` at a variable (`set $backend "http://calculator:8080"`) and declare `resolver 127.0.0.11 valid=10s`, which forces re-resolution against Docker's embedded DNS per request instead of at startup.

8 — A catch-all handler that turned 404 into 500. `@ExceptionHandler(Exception.class)` on a `@RestControllerAdvice` is consulted *before* Spring's `DefaultHandlerExceptionResolver`, so it also swallowed the framework's own `NoResourceFoundException` and `HttpRequestMethodNotSupportedException`: an unknown path answered 500 instead of 404, a wrong method 500 instead of 405, and each one logged a full stack trace at ERROR. Both are now handled explicitly, with integration tests pinning the status codes.

9 — `Infinity` leaking through the JSON contract. Rejecting `b = 0` was not enough: `1e308 / 1e-308` still overflowed, and Jackson cannot encode a non-finite `double` as a JSON number, so it silently emitted `"result":"Infinity"` — a *string* where every client expects a number. Operands and results are now both checked with `Double.isFinite` and rejected as 400, which is also what makes the "validate numeric inputs" requirement genuinely met rather than only true for the divide-by-zero case.

8. Outcome

A single `git push` now produces: a compiled and tested artefact, a published Docker Hub image tagged with the build number, a redeployed two-replica stack behind Nginx, an automated health and smoke-test gate, and an email with the result. The project also forms the base for the end-of-semester Capstone, which will extend it with Ansible, Kubernetes and Prometheus/Grafana.

Verified locally. `./mvnw clean test` → 35 tests, 0 failures. `docker build` → a ~230 MB runtime image running as uid 10001. `docker compose up -d` → two API replicas and the proxy all `(healthy)`, with Nginx observed splitting requests across both replicas, and a second `up -d` confirmed to be a no-op. `./scripts/smoke-test.sh` → 12/12 checks pass against the live stack, covering every 400 path plus the 404 and 405 cases.

Evidence to be attached (`docs/screenshots/`, checklist in that directory): a green Jenkins run with all seven stages, the build-status email, `docker ps` showing both replicas and the proxy as `healthy`, the Docker Hub tag list, and the GitHub webhook delivery. These depend on the Jenkins instance and the GitHub remote, and are captured as part of preparing the Class 17 demo.